

Why Does Self-Distillation (Sometimes) Degrade the Reasoning Capability of LLMs?

Jeonghye Kim^{1,2*}, Xufang Luo^{1†}, Minbeom Kim³, Sangmook Lee³,
Dohyung Kim³, Jiwon Jeon², Dongsheng Li¹, Yuqing Yang¹

¹Microsoft Research ²KAIST ³Seoul National University

[blog post](#) [beanie00/self-distillation-analysis](#)

Abstract

Self-distillation has emerged as an effective post-training paradigm for LLMs, often improving performance while shortening reasoning traces. However, in mathematical reasoning, we find that it can reduce response length while degrading performance. We trace this degradation to the suppression of **epistemic verbalization**—the model’s expression of uncertainty during reasoning. Through controlled experiments varying **conditioning context richness** and **task coverage**, we show that conditioning the teacher on rich information suppresses uncertainty expression, enabling rapid in-domain optimization with limited task coverage but harming OOD performance, where unseen problems benefit from expressing uncertainty and adjusting accordingly. Across Qwen3-1.7B/8B, DeepSeek-Distill-Qwen-7B, and Olmo3-7B-Instruct, we observe performance drops of up to 40%. Our findings highlight that exposing appropriate levels of uncertainty is crucial for robust reasoning and underscore the importance of optimizing reasoning behavior beyond merely reinforcing correct answer traces.

1 Introduction

Recently, self-distillation (Snell et al., 2022) has attracted increasing attention in the post-training of large language models (LLMs). In this paradigm, two instances of the same model are employed: one conditioned on the ground-truth solutions serves as a teacher, providing informative reward signals for responses generated by another instance that does not have access to the solutions. Several studies have demonstrated that combining this framework with post-training methods such as Reinforcement Learning from Verifiable Rewards (RLVR) leads to highly efficient performance gains (Zhu et al., 2025; Kim et al., 2026; Hübottter et al., 2026; Shenfeld et al., 2026; Song et al., 2026; Zhao et al., 2026; Ye et al., 2026). These methods have shown particularly strong improvements in domains such

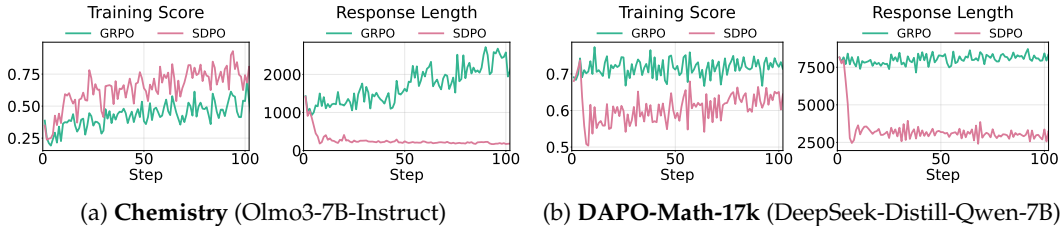


Figure 1: (a) Training score and response length changes for GRPO and Self-Distillation (SDPO) (Hübottter et al., 2026) in Chemistry, using results from SDPO Wandb logs (Biewald, 2020) (link). (b) Training score and response length changes on DAPO-Math-17k with GRPO and SDPO.

^{0*} Work done during an internship at Microsoft Research.

^{0†} Corresponding author.

as agentic environments and scientific reasoning, especially under in-domain evaluation settings. Interestingly, a consistent trend observed across these works is that performance improves as response length decreases, suggesting that self-distillation promotes more concise and effective reasoning.

However, when we apply the same self-distillation approach to mathematical reasoning tasks, we observe a markedly different phenomenon. Figure 1 compares the effects of a representative self-distillation algorithm, SDPO, in the Chemistry domain (a) and the Math domain (b). As shown in the figure, in the Chemistry domain, self-distillation substantially reduces response length compared to GRPO while rapidly improving performance. In contrast, in the Math domain, although response length consistently decreases as training progresses, performance drops significantly, contrary to prior findings.

This raises a question: *“Why does performance sometimes degrade despite the model being trained to move toward the correct answer?”*

Our analysis reveals a consistent pattern: the more informative the context provided to the teacher, the more concise and confident the resulting reasoning becomes, with substantially fewer expressions of uncertainty and, particularly in math reasoning, degraded performance. We trace this effect to the suppression of *epistemic verbalization* (Kim et al., 2026), whereby models explicitly verbalize and incorporate uncertainty during reasoning. Strong reasoning models such as DeepSeek-R1 (Guo et al., 2025b) frequently express uncertainty using tokens like “Wait” or “Hmm”. Although these expressions may not directly advance the reasoning, removing them discards important signals that a reasoning path may be flawed, leading to significant performance drops (Kim et al., 2026).

To systematically understand when and why self-distillation suppresses epistemic verbalization, we conduct a comprehensive empirical study and identify two key factors: **information richness** and **task coverage**. When the teacher is conditioned on richer information, such as the correct solution, it produces reasoning trajectories with little expressed uncertainty, encouraging the student to imitate a confident reasoning style that presupposes information unavailable at inference time. When task coverage is limited, this compression enables rapid in-domain optimization. However, as coverage increases, the trained removal of epistemic verbalization can interfere with optimization across diverse tasks, degrading performance on more challenging or previously unseen problems.

More broadly, our results show that even when the training objective faithfully guides the model toward correct reasoning traces, the resulting reasoning style can quietly shift in ways that hurt generalization. The suppression of epistemic verbalization is not penalized by standard objectives, yet negatively impacts out-of-distribution (OOD) performance. This suggests that post-training objectives need to account not only for answer correctness, but also for eliciting and preserving uncertainty-aware reasoning behaviors. We believe these findings offer a useful step toward a deeper understanding of reasoning in self-distillation and post-training more broadly.

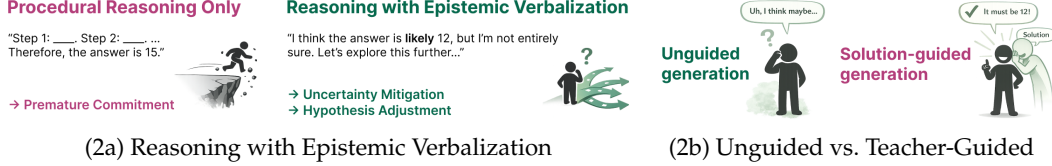
2 Preliminaries

Self-Distillation Let $x \in \mathcal{X}$ denote an input and $y = (y_1, \dots, y_T)$ a sequence generated by a language model π_θ . The model defines an autoregressive distribution $\pi_\theta(y|x) = \prod_{t=1}^T \pi_\theta(y_t | x, y_{<t})$. In self-distillation, the same model acts as both a *student* and a *teacher* under different conditioning contexts. The student first generates a sequence $y \sim \pi_\theta(\cdot | x)$. The teacher policy is obtained by conditioning the model on a *richer context* c that provides additional information about the input (e.g., solutions, environment feedback, or other auxiliary signals): $\pi_\theta^T(\cdot | x, c) = \pi_\theta(\cdot | x, c)$. Training minimizes the divergence between the student and teacher next-token distributions:

$$\mathcal{L}_{\text{SD}}(\theta) = \sum_t \text{KL}(\pi_\theta(\cdot | x, y_{<t}) \parallel \text{stopgrad}(\pi_\theta(\cdot | x, c, y_{<t}))). \quad (1)$$

This objective encourages the student to match the teacher’s predictions under the richer context, enabling the model to improve by distilling information available at training time without requiring an external teacher.

Key Characteristics of Math Reasoning In LLMs, math reasoning can be viewed as *self-Bayesian reasoning*, where each step is generated conditioned only on the problem and previously generated tokens, with the model iteratively updating its belief over intermediate hypotheses (Kim et al., 2026). At the same time, math reasoning spans diverse tasks such as arithmetic, algebra, geometry, word problems, and logical pattern recognition, making evaluation benchmarks frequently OOD relative to training data due to compositional and reasoning-depth shifts.



Within this process, verbalized uncertainty toward y —referred to as *epistemic verbalization* (Kim et al., 2026)—can serve as an informative signal rather than mere stylistic redundancy. As illustrated in Figure 2(2a), reasoning without such signals may lead the model to prematurely commit to incorrect hypotheses with limited opportunity for recovery, whereas epistemic verbalization helps maintain alternative hypotheses and supports gradual uncertainty reduction.

In self-distillation, the teacher has access to a richer context c , enabling it to generate reasoning trajectories with strong hints and minimal expressed uncertainty. While this leads to more concise responses, it may hinder the student’s ability to perform uncertainty-aware reasoning. Consequently, aggressive length constraints and overly confident reasoning styles risk eliminating not only unnecessary verbosity but also valuable epistemic signals, especially in smaller models with limited parametric knowledge. We systematically analyze how the richness of the conditioning context relates to the suppression of uncertainty expression as well as reasoning performance, and investigate: if epistemic verbalization is necessary, why does suppressing it lead to better performance in domains such as Chemistry while degrading it in others? A deeper analysis of this discrepancy and the role of task coverage is provided in Section 6.

3 LLM Reasoning Behavior Under Richer Information

Before analyzing self-distillation in depth, we examine how LLM reasoning changes when richer information is provided. To formalize the informativeness of the conditioning context, we define the information that c provides about the target sequence y as the conditional mutual information

$$I(y; c \mid x) = H(y \mid x) - H(y \mid x, c), \quad (2)$$

which captures the reduction in uncertainty about y once the additional context c is given.

Using the DAPO-Math-17k dataset (Yu et al., 2025) and DeepSeek-R1-Distill-Qwen-7B (Guo et al., 2025b) as the base model, we select 100 problems on which the base model achieves accuracy between 0.125 and 0.5 over 8 rollouts. Let s denote the full solution (including chain-of-thought in `<think>` tags), $s_{\setminus \text{think}}$ the solution with `<think>` content removed, and $\tilde{y}$ a response previously generated under full solution guidance. We compare the model’s responses across four generation settings with increasing conditioning information:

- **(1) Unguided generation:** $c = \emptyset$, so $I(y; c \mid x) = 0$ by definition.
- **(2) Solution-guided generation:** $c = s$, providing maximal guidance and yielding the largest $I(y; c \mid x)$.
- **(3) Solution-guided generation (without think contents):** $c = s_{\setminus \text{think}}$. Since $s_{\setminus \text{think}}$ is a strict informational subset of s , we have $I(y; s_{\setminus \text{think}} \mid x) \leq I(y; s \mid x)$.
- **(4) Regeneration-conditioned generation:** $c = \tilde{y}$, where $\tilde{y}$ is generated under setting (2). By the data processing inequality, $I(y; \tilde{y} \mid x) \leq I(y; s \mid x)$.

These settings induce the following ordering over the conditional mutual information:

$$\underbrace{I(y; c \mid x)}_{(1)} = 0 < \underbrace{I(y; s_{\setminus \text{think}} \mid x)}_{(3)} \leq \underbrace{I(y; \tilde{y} \mid x)}_{(4)} \leq \underbrace{I(y; s \mid x)}_{(2)}. \quad (3)$$

Prompts The prompts used for unguided and solution-guided settings are as follows. For regeneration, we used the same prompts as in Hübötter et al. (2026). A comparative analysis with other regeneration prompts is provided in Appendix C.

Prompt for unguided generation	{question} Please reason step by step, and put your final answer within \boxed{ }.
Regeneration prompt (followed the prompt in Hübötter et al. (2026))	{question} Please reason step by step, and put your final answer within \boxed{ }. Correct solution: {previously correct solution} Correctly solve the original question.

Epistemic tokens Following Kim et al. (2026), we define a set of 10 epistemic markers $\mathcal{T} = \{\text{wait, hmm, perhaps, maybe, actually, alternatively, seems, might, likely, check}\}$ as practical indicators of regions where the externalization of uncertainty is likely to occur. These tokens are intuitively recognizable as uncertainty expressions, and we further verify that they co-occur with extended uncertainty expressions in sampled LLM outputs (see Table 4 in Appendix B.5). We measure the epistemic token count of a response y as $E(y) = \sum_{t \in \mathcal{T}} \text{count}(t, y)$.

Results We analyze how different forms of solution guidance affect the model’s reasoning behavior by comparing the average response length $\mathbb{E}[L(y)]$, model score, and the epistemic token count $\mathbb{E}[E(y)]$ across the four settings. As shown in Table 1, both quantities decrease monotonically as $I(y; c \mid x)$ increases:

$$\mathbb{E}[L(y)] \Big|_{(1)} > \mathbb{E}[L(y)] \Big|_{(3)} > \mathbb{E}[L(y)] \Big|_{(4)} > \mathbb{E}[L(y)] \Big|_{(2)}, \quad (4)$$

and analogously for $\mathbb{E}[E(y)]$, confirming that richer conditioning information leads to more concise and confident reasoning.

Specifically, unguided generation ($c = \emptyset$) produces substantially longer responses with the highest epistemic token counts. When the full solution s is provided in (2), the model follows the given reasoning trajectory with high confidence, and its concise output can be viewed as a compressed representation of the essential reasoning in s . In (3), removing the `<think>` portion retains only $s_{\setminus \text{think}}$ (640 out of 13,054 response tokens), and both $\mathbb{E}[L(y)]$ and $\mathbb{E}[E(y)]$ increase again toward the unguided level, reflecting the substantial information loss. Setting (4), conditioning on the regenerated response $\tilde{y}$, yields intermediate values—lower than (3) but higher than (2)—indicating that $\tilde{y}$ preserves much of the informative structure of the full solution. Detailed per-token breakdowns are reported in Appendix B.1.

Table 1: Comparison of response characteristics under varying levels of rich information.

	Avg. Score	Avg. Length	Epistemic Token Count
(1) Unguided	0.30	13,054	182.5
(2) Solution-Guided ($c = s$)	0.98	1,873	8.8
(3) Solution-Guided ($c = s_{\setminus \text{think}}$)	0.78	12,036	159.8
(4) Regeneration-Conditioned	0.95	2,808	24.1

Takeaway 1: Information Richness and Epistemic Verbalization

As the conditioning context c becomes more informative and directly useful, the LLM generates answers more confidently and with fewer expressions of epistemic uncertainty.

4 Supervised Finetuning with Self-Distillation

A natural follow-up question is whether the suppression of epistemic verbalization under high $I(y; c \mid x)$ is merely stylistic or has a tangible impact on reasoning capability. To test this, we conduct off-policy self-distillation (SFT) using DeepSeek-R1-Distill-Qwen-7B (Guo et al., 2025a) on two datasets, each containing 800 correct responses:

- $\mathcal{D}_{\text{ug}}$: unguided responses ($c = \emptyset$), with high $\mathbb{E}[E(y)]$ and $\mathbb{E}[L(y)] \approx 12\text{k}$ tokens.
- $\mathcal{D}_{\text{sg}}$: solution-guided responses ($c = s$), with low $\mathbb{E}[E(y)]$ and $\mathbb{E}[L(y)] \approx 2\text{k}$ tokens.

Both datasets consist entirely of correct trajectories; the key difference lies in the epistemic density of the training signal. We evaluate the resulting checkpoints across multiple math benchmarks (examples from each dataset are presented in Appendix H.1).

Table 2: Math benchmark performance of the base model DeepSeek-R1-Distill-Qwen-7B and its SFT checkpoints trained on unguided and solution-guided datasets.

DeepSeek-R1-Distill-Qwen-7B	AIME24	AIME25	AMC23	MATH500
Base	54.79	37.92	89.06	92.19
SFT on $\mathcal{D}_{\text{ug}}$	51.04	40.00	87.66	90.93
SFT on $\mathcal{D}_{\text{sg}}$	20.21	12.71	57.03	65.52

As shown in Table 2, training on $\mathcal{D}_{\text{sg}}$ leads to substantial degradation across all benchmarks, despite the dataset consisting of correct answers, whereas training on $\mathcal{D}_{\text{ug}}$ produces no significant performance change. This asymmetry arises because solution-guided responses are concise precisely due to the external context s ; using them as SFT targets *without* s forces the model to imitate a reasoning style that presupposes information unavailable at inference time, effectively suppressing the epistemic tokens that support autonomous exploration and error correction.

Takeaway 2: Epistemic Suppression and Reasoning Performance

Even when trained on correct trajectories, excessively suppressing epistemic verbalization can substantially degrade reasoning performance.

5 On-Policy Self-Distillation

We now turn to on-policy self-distillation (Hübotter et al., 2026; Zhao et al., 2026; Ye et al., 2026), where the model learns from dense reward signals provided by a self-teacher with access to the correct solution, based on its own responses. We compare GRPO with Reinforcement Learning via Self-Distillation (SDPO) (Hübotter et al., 2026) on the DAPO-Math-17k dataset (Yu et al., 2025), using Qwen3-8B (Yang et al., 2025) and DeepSeek-R1-Distill-Qwen-7B (Guo et al., 2025b). Additional results for Qwen3-1.7B and Olmo-3-7B-Instruct (Olmo et al., 2025) are in Appendix F.3. For each model, we track training score, response length, and OOD performance on AIME24 and AMC23. We fix the teacher to the initial policy, which performs better than a moving target (see Section 5.4 for a comparison).

The behavior of on-policy self-distillation depends on two factors: (i) how much the base model already shows epistemic verbalization, and (ii) the richness of the conditioning context c . To disentangle these, we compare GRPO and SDPO under two settings: $c = s$ (full solution) and $c = s_{\setminus \text{think}}$ (solution without <think> content).

5.1 DeepSeek-R1-Distill-Qwen-7B

DeepSeek-R1-Distill-Qwen-7B is a representative high-reasoning model that generates rich epistemic verbalizations within <think> tags, averaging about 180 epistemic tokens per response, as shown in Section 3, leading to lengthy yet high-performing reasoning outputs.

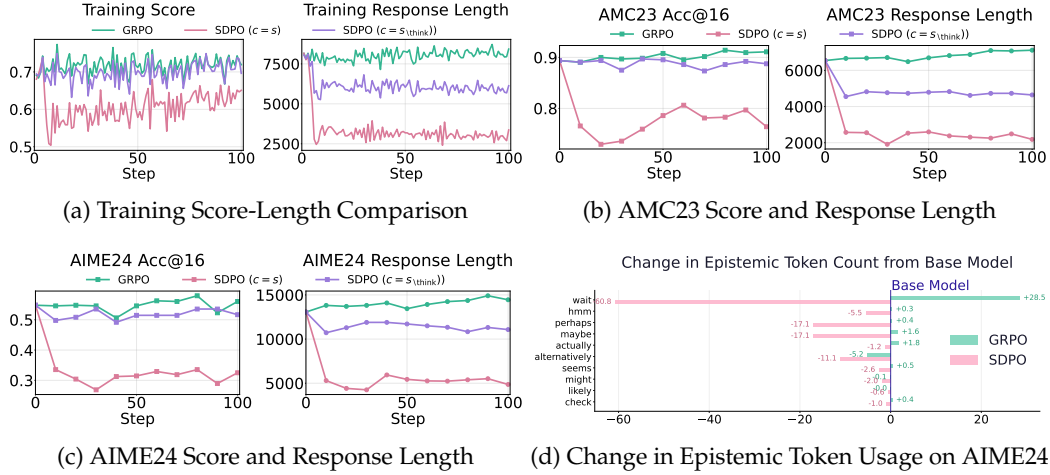


Figure 3: On-policy self-distillation results for DeepSeek-R1-Distill-Qwen-7B. GRPO yields modest OOD gains with a slight increase in epistemic verbalization, whereas SDPO degrades both performance and epistemic token usage, particularly with $c = s$.

Training Performance As shown in Figure 3a, GRPO training slightly increases $\mathbb{E}[L(y)]$ with a modest improvement in score. In contrast, SDPO with $c = s$ causes a sharp initial drop in both $\mathbb{E}[L(y)]$ and score; performance gradually recovers but remains below GRPO throughout training. When the conditioning is reduced to $c = s_{\text{think}}$, the drop in $\mathbb{E}[L(y)]$ is attenuated and the score trajectory approaches that of GRPO, consistent with the relationship between $I(y; c | x)$ and epistemic suppression discussed in Section 3.

OOD Evaluation - AIME24, AMC23 Consistent with the training trends, GRPO yields modest gains on both OOD benchmarks (AIME24: $54.7 \rightarrow 56.0$; AMC23: $89.3 \rightarrow 91.1$, Figures 3b and 3c) with a slight increase in $\mathbb{E}[L(y)]$. SDPO with $c = s$ degrades performance substantially ($\sim 40\%$ on AIME24, $\sim 15\%$ on AMC23). Reducing the conditioning to $c = s_{\text{think}}$ mitigates the drop, though performance still remains below the base model.

Reasoning Pattern Figure 3d illustrates the epistemic token counts of the trained models. GRPO increases $\mathbb{E}[E(y)]$, whereas SDPO suppresses it more aggressively, consistent with the observed correlation between epistemic suppression and performance degradation throughout our analysis. Additional analyses on total versus epistemic token changes, as well as the number of uncertainty expressions per reasoning trace, evaluated using an LLM-as-Judge, are provided in Appendices B.4 and B.5. Furthermore, a comparison of the KL divergence of epistemic tokens with respect to the mean is presented in Appendix B.6. Example responses generated by GRPO and SDPO are included in Appendix H.2.

5.2 Qwen3-8B (Thinking Mode: ON)

Qwen3-8B is a hybrid reasoning model that can operate in either a thinking mode for deliberate reasoning or a non-thinking mode for direct responses. With thinking mode enabled, Qwen3-8B produces longer responses than DeepSeek-R1-Distill-Qwen-7B, with a higher number of epistemic tokens (see Appendix B.2 for a comparison of epistemic tokens across models).

Training Performance As shown in Figure 4a, $\mathbb{E}[L(y)]$ decreases under both GRPO and SDPO, with SDPO exhibiting a larger reduction and a correspondingly larger performance drop. Notably, $\mathbb{E}[L(y)]$ first drops sharply, then increases slightly. Since the teacher policy is fixed as the reference policy, shortening the response by ~ 900 tokens reduces the informativeness of c , i.e., decreases $I(y; c | x)$. As the context becomes less informative, the model compensates by increasing epistemic verbalization, causing the length to partially recover.

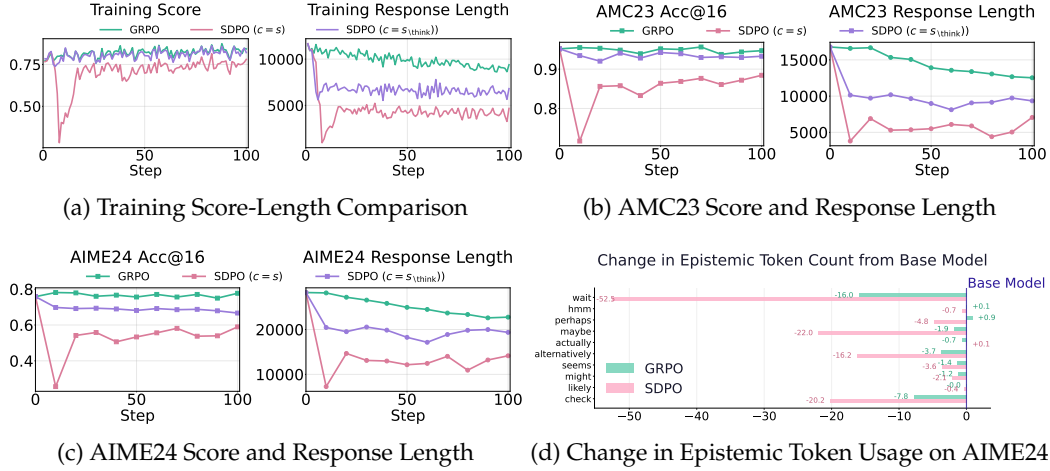


Figure 4: On-policy self-distillation results for Qwen3-8B (Thinking Mode: ON). Both GRPO and SDPO reduce response length and epistemic verbalization, but SDPO’s more aggressive suppression leads to greater degradation in OOD performance, particularly on AIME24.

OOD Evaluation - AIME24, AMC23 The gap becomes more pronounced on OOD benchmarks (Figures 4b and 4c): GRPO maintains largely stable performance with gradually decreasing $\mathbb{E}[L(y)]$, whereas SDPO falls below the base model, particularly with $c = s$. Notably, although GRPO and SDPO with $c = s_{\text{think}}$ achieve comparable training performance, their OOD results diverge, especially on the more challenging AIME24, where SDPO with $c = s_{\text{think}}$ shows progressive performance degradation as training proceeds.

Reasoning Pattern Figure 4d reveals that both methods reduce $\mathbb{E}[E(y)]$ relative to the base model, with SDPO more aggressively so. This suggests that Qwen3-8B originally generates more epistemic verbalization than necessary; while both methods mitigate this redundancy, overly aggressive suppression risks removing epistemic signals that carry useful reasoning information.

5.3 Qwen3-8B (Thinking Mode: OFF)

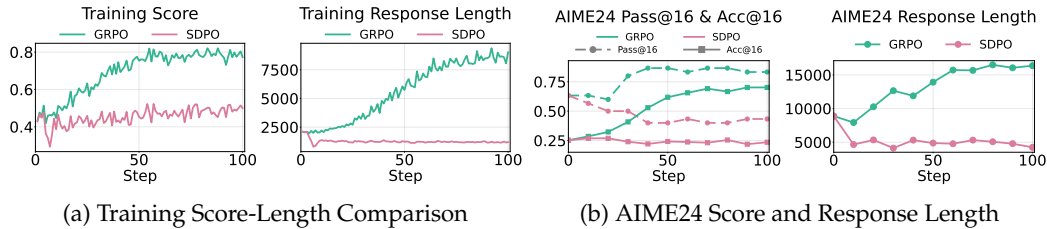


Figure 5: On-policy self-distillation results for Qwen3-8B (Thinking Mode: OFF). GRPO rapidly increases response length via epistemic verbalization and achieves strong training gains, while SDPO reduces response length and struggles to improve, with slight OOD degradation on AIME24.

When Qwen3-8B is used without thinking mode, the `<think>` tag is absent, so we consider only $c = s$. The model initially produces much shorter responses and exhibits significantly lower performance. GRPO rapidly increases $\mathbb{E}[L(y)]$ by promoting epistemic verbalization (as shown in Appendix F.1), quickly achieving a high training score. In contrast, SDPO reduces $\mathbb{E}[L(y)]$ and improves much more slowly; even when the training score slightly increases, as shown in Figure 5b, performance on AIME24 slightly declines ($0.25 \rightarrow 0.23$), further illustrating the cost of epistemic suppression under self-distillation.

5.4 Ablation Study: Fixed vs. Moving Target Teacher

In naive on-policy self-distillation, the teacher and student share a continuously updated policy, making the teacher a moving target that can introduce training instability (Zhao et al., 2026; Ye et al., 2026). To mitigate this, SDPO uses an EMA-smoothed teacher (EMA rate: 0.05). However, we find that setting the EMA rate to 0.0 (i.e., fixing the teacher to the initial policy) yields better performance, and thus Section 5 follows this setting.

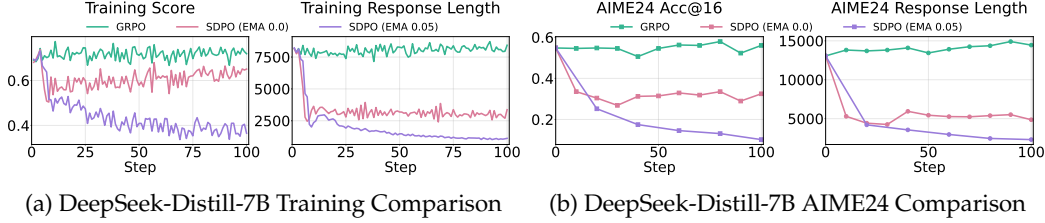


Figure 6: Fixed vs. moving target teacher for DeepSeek-R1-Distill-Qwen-7B. Even slow EMA updates (rate 0.05) amplify epistemic suppression via a feedback loop, causing greater performance degradation than a fixed teacher.

Figure 6a shows additional comparison results when the teacher is updated during training. As shown, even slow updates (e.g., rate 0.05) lead to a sharper reduction in response length, resulting in larger performance degradation. This can be interpreted as a feedback loop in self-distillation: the model is trained to produce increasingly confident outputs, and when a checkpoint of the same model is used as the teacher, it generates even more confident responses, amplifying the effect over iterations. Further ablations on learning rate and top-K logits are in Appendix G.

Takeaway 3: Epistemic Verbalization Changes and Performance in On-Policy Self-Distillation

As the teacher context c becomes more information-rich, on-policy self-distillation reduces epistemic verbalization and shortens responses. This effect varies depending on the level of uncertainty originally expressed by the base model.

6 Relationship Between Task Coverage, Epistemic Verbalization and Generalization Ability

Across the off-policy and on-policy settings analyzed above, self-distillation consistently produces more confident responses with reduced $\mathbb{E}[E(y)]$. This aligns with the findings of Hübötter et al. (2026), which reports that SDPO learns to reason concisely: on Science Q&A (Chemistry, Physics, Biology, and Materials Science) (Feng et al., 2024), tool use (Tang et al., 2023), and LiveCodeBench v6 (Jain et al., 2025), SDPO achieves higher accuracy than GRPO while producing substantially shorter outputs with fewer epistemic markers.

In other words, in these domains, self-distillation suppresses epistemic verbalization and improves performance simultaneously. The key question is *why the same mechanism leads to performance degradation in our math-focused setup*. We hypothesize that the answer lies in differences in task coverage between the training and evaluation distributions.

6.1 Comparison of Task Coverage

To test this hypothesis, we compare the dataset characteristics of the settings where SDPO outperformed GRPO against our experimental setup. As shown in Table 3, the Chemistry dataset, despite its large size, draws from only six main problem types that differ primarily in surface details rather than underlying structure. LiveCodeBench v6 contains diverse problems but only 131 in total, leading to repeated exposure during training with identical train/eval splits. In contrast, DAPO-Math-17k exposes the model to 14,000 distinct problems (78% of the 25,600 samples drawn over 100 steps, due to repeated sampling), spanning a

broad, non-overlapping range of problem types, and evaluation is performed on unseen problem types.

Table 3: Comparison of the total number of problems, problem composition, and train–evaluation splits in ScienceQ&A, LiveCodeBench v6, and our experiments.

Domain	Dataset Analysis
ScienceQ&A (Chemistry)	Total 2,400 questions: Reaction Balancing (300), Molecular Descriptor Counting (300), Molecular Weight Calculation (600), Property Prediction (e.g., logS) (500), Precursor/Reactant Selection (Retrosynthesis) (300), Product Prediction (Organic Reactions) (400). <i>Train/Eval split</i> : 90% for training, 10% for evaluation.
LiveCodeBench v6	<i>Train/Eval split</i> : Total 131 questions for training and all 131 questions are used for both training and evaluation. Only 50% of the public test cases are used during training, while the full set including hidden test cases is used for evaluation.
DAPO-Math-17k	<i>Train/Eval split</i> : Total 14,000 questions for training and evaluation is conducted on standard benchmarks (AIME 2024/2025, AMC 2023, MATH 500) using questions not part of the training data.

6.2 Relationship Between Task Coverage and Learning Performance

To further investigate the interplay between task coverage and generalization, we vary the number of training questions $|\mathcal{D}| \in \{1, 8, 64, 128, 512\}$ from DAPO-Math-17k and train with both GRPO and SDPO. All experiments use Qwen3-8B (Thinking Mode OFF).

Training Logs As shown in Figure 7, GRPO and SDPO exhibit distinct training dynamics as $|\mathcal{D}|$ varies. When $|\mathcal{D}| \leq 128$, SDPO quickly achieves high scores while reducing $\mathbb{E}[L(y)]$ by up to $8\times$, indicating higher training efficiency on a small task set. However, at $|\mathcal{D}| = 512$, further reductions in $\mathbb{E}[L(y)]$ begin to hurt the training score relative to GRPO, whose $\mathbb{E}[L(y)]$ gradually increases with $|\mathcal{D}|$.

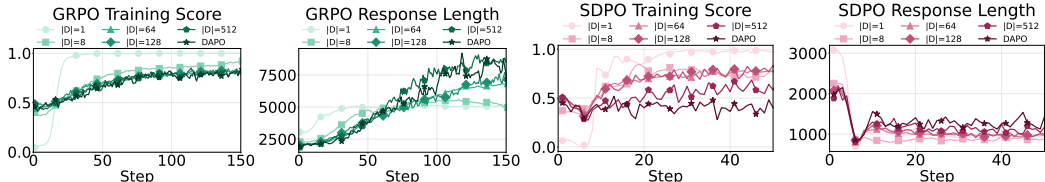


Figure 7: Training score and response length comparison between GRPO and SDPO for $|\mathcal{D}| \in \{1, 8, 64, 128, 512\}$.

This difference can be interpreted through **task coverage**. As $|\mathcal{D}|$ grows, the model must accommodate a broader range of reasoning patterns. GRPO addresses this by increasing $\mathbb{E}[E(y)]$, allowing the model to express greater uncertainty and adapt its reasoning accordingly. SDPO instead encourages confident, concise responses—**effective when task coverage is small but limiting when the problem set becomes larger and more diverse**.

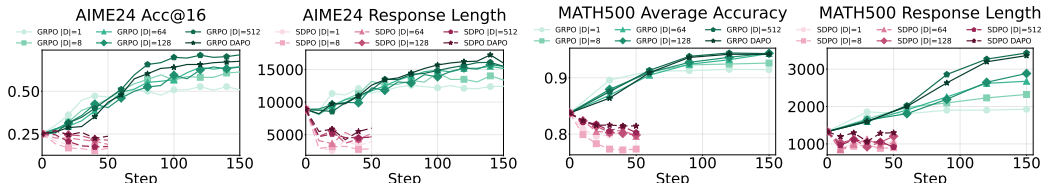


Figure 8: Evaluation performance on AIME24 and MATH500 and response length as $|\mathcal{D}|$ varies over $\{1, 8, 64, 128, 512\}$.

OOD Evaluation - AIME24, MATH500 The distinction between GRPO and SDPO becomes more pronounced on OOD benchmarks (Figure 8). Under GRPO, performance scales consistently with $|\mathcal{D}|$: $|\mathcal{D}| = 1$ converges quickly but soon stops improving, while

larger $|\mathcal{D}|$ yields progressively higher final scores accompanied by increasing $\mathbb{E}[L(y)]$. Under SDPO, the pattern reverses: smaller $|\mathcal{D}|$ leads to more severe OOD degradation. Even at the largest $|\mathcal{D}|$ (DAPO setting), SDPO still underperforms the base model. Changes in GRPO and SDPO reasoning patterns across task coverage are provided in Appendix B.3.

Takeaway 4: Epistemic Verbalization Depends on Task Generalization

The value of epistemic verbalization scales with generalization demands: largely redundant for familiar, repetitive tasks (small $|\mathcal{D}|$) and removable for efficiency, but increasingly important as task diversity grows.

7 Conclusion

In this work, we provide an empirical analysis of on-policy self-distillation, motivated by an information-theoretic view of conditioning context richness. Our experiments suggest that the effectiveness of self-distillation is closely tied to how information is provided to the model and how the model expresses uncertainty during reasoning. We observe that self-distillation tends to produce answers with higher confidence and shorter reasoning traces. While this effect enables more compact reasoning and can quickly improve in-domain performance when task coverage is limited, it becomes less effective when task coverage is broad and may even harm OOD performance.

Importantly, the fact that performance can degrade even when a mathematically sound objective function is designed to elicit correct CoT reasoning suggests that the choice of optimization objective alone may not be sufficient for preserving robust reasoning, and that we need to pay closer attention to how training reshapes the model’s reasoning behavior, beyond answer correctness. We hope that our analysis contributes to a deeper understanding of self-distillation and other LLM post-training methods, and provides insights for developing more robust training strategies.

Acknowledgments

We thank Li Dong, Tianzhu Ye, and Sojeong Rhee for their valuable discussions. This work was supported by Microsoft Research and partly by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT)(RS-2025-00557589).

Ethics Statement

This work aims to analyze the causes of reasoning performance degradation during self-distillation of LLMs. All experiments were conducted using publicly available models (Qwen3, DeepSeek-R1-Distill-Qwen, OLMo) and datasets (DAPO-Math-17k, AIME, AMC, MATH500). As this study does not involve any new data collection or human subjects research, concerns related to privacy or human subjects ethics do not apply.

A key finding of this work is that self-distillation can suppress epistemic verbalization, thereby diminishing a model’s ability to express uncertainty during reasoning. This observation carries important implications for the design of LLM post-training methods. The ability to appropriately express uncertainty is directly related to model reliability and safety; training procedures that indiscriminately suppress such expressions may increase the risk of models producing incorrect answers with unwarranted confidence. We hope that our analysis contributes to the development of more robust and trustworthy reasoning models.

Disclosure of LLM Usage

We used LLMs for conducting the LLM-as-a-Judge experiments described in Appendix B.5, refining data plot designs, and improving the clarity of our writing. LLMs were not involved in research ideation, and their role in writing was limited to polishing author-written text.

References

- Lukas Biewald. Experiment tracking with weights and biases, 2020. URL <https://www.wandb.com/>. Software available from wandb.com.
- Jie Deng, Shining Liang, Jun Li, Hongzhi Li, and Yutao Xie. Conpress: Learning efficient reasoning from multi-question contextual pressure. *arXiv preprint arXiv:2602.01472*, 2026.
- Kehua Feng, Xinyi Shen, Weijie Wang, Xiang Zhuang, Yuqi Tang, Qiang Zhang, and Keyan Ding. Sciknoweval: Evaluating multi-level scientific knowledge of large language models. *arXiv preprint arXiv:2406.09098*, 2024.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL <https://zenodo.org/records/12608602>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025a.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025b.
- Jonas Hübner, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, et al. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=chfJJYC3iL>.
- Jeonghye Kim, Xufang Luo, Minbeom Kim, Sangmook Lee, Dohyung Kim, Dongsheng Li, and Yuqing Yang. Understanding reasoning in llms through strategic information allocation under uncertainty, 2026.
- Zeyuan Liu, Jeonghye Kim, Xufang Luo, Dongsheng Li, and Yuqing Yang. Exploratory memory-augmented LLM agent via hybrid on- and off-policy optimization. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=U0zxviKVFO>.
- Qin-Wen Luo, Sheng Ren, Xiang Chen, Rui Liu, Jun Fang, Naiqiang Tan, and Sheng-Jun Huang. Compress the easy, explore the hard: Difficulty-aware entropy regularization for efficient llm reasoning. *arXiv preprint arXiv:2602.22642*, 2026.
- Team Olmo, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heine-man, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, et al. Olmo 3. *arXiv preprint arXiv:2512.13961*, 2025.
- Emiliano Penalzoza, Dheeraj Vattikonda, Nicolas Gontier, Alexandre Lacoste, Laurent Charlin, and Massimo Caccia. Privileged information distillation for language models. *arXiv preprint arXiv:2602.04942*, 2026.
- Hejian Sang, Yuanda Xu, Zhengze Zhou, Ran He, Zhipeng Wang, and Jiachen Sun. On-policy self-distillation for reasoning compression. *arXiv preprint arXiv:2603.05433*, 2026.
- Idan Shenfeld, Mehul Damani, Jonas Hübner, and Pulkit Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.

- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- Vaishnavi Shrivastava, Ahmed Hassan Awadallah, Vidhisha Balachandran, Shivam Garg, Harkirat Behl, and Dimitris Papailiopoulos. Sample more to think less: Group filtered policy optimization for concise reasoning. In *First Workshop on Foundations of Reasoning in Language Models*, 2025.
- Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- Charlie Snell, Dan Klein, and Ruiqi Zhong. Learning by distilling context. *arXiv preprint arXiv:2209.15189*, 2022.
- Yuda Song, Lili Chen, Fahim Tajwar, Remi Munos, Deepak Pathak, J Andrew Bagnell, Aarti Singh, and Andrea Zanette. Expanding the capabilities of reinforcement learning via text feedback. *arXiv preprint arXiv:2602.02482*, 2026.
- Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*, 2023.
- Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformers Reinforcement Learning, 2020. URL <https://github.com/huggingface/trl>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Zhicheng Yang, Zhijiang Guo, Yinya Huang, Yongxin Wang, Wenlei Shi, Yiwei Wang, Xiaodan Liang, and Jing Tang. Accordion-thinking: Self-regulated step summaries for efficient and readable llm reasoning. *arXiv preprint arXiv:2602.03249*, 2026.
- Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*, 2026.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.
- Mingye Zhu, Yi Liu, Zheren Fu, Quan Wang, and Yongdong Zhang. In-token rationality optimization: Towards accurate and concise llm reasoning via self-feedback. *arXiv preprint arXiv:2511.09865*, 2025.

A Related Works

Self-Distillation for LLM Reasoning. Self-distillation, where a single model acts as both teacher and student under asymmetric conditioning (Snell et al., 2022), has recently been widely adopted for LLM post-training. SDPO (Hübotter et al., 2026) conditions the self-teacher on its previously generated correct trajectories or rich environment feedback to provide dense credit assignment, while OPSD (Zhao et al., 2026) leverages ground-truth solutions as privileged information, achieving strong efficiency gains in both cases. Our analysis builds upon the SDPO codebase, and a detailed comparison with OPSD is provided in Appendix E. Penaloza et al. (2026), Liu et al. (2026), and Ye et al. (2026) further extend this paradigm to settings that incorporate additional signals, such as privileged information, self-generated hints derived from environmental interactions, or contextual guidance, with the goal of internalizing this external knowledge into the model’s parameters.

While these works demonstrate the effectiveness of self-distillation, they do not investigate when or why self-distillation may degrade reasoning performance, especially in scenarios where a model must solve problems entirely on its own, without interaction with the external environment, which is the central question of our work.

Uncertainty Expression in LLM Reasoning. Kim et al. (2026) introduce an information-theoretic framework that identifies epistemic verbalization, the explicit externalization of uncertainty, as informationally necessary for correct reasoning. Our work answers a question this framework cannot readily explain: *if epistemic verbalization is necessary, why does suppressing it lead to better performance in some domains such as Chemistry while degrading it in others?* We trace this to two factors: **conditioning context richness** (Section 3) and **task coverage** (Section 6), supported by a systematic analysis of recent on-policy self-distillation methods across various models with varying pretrained capabilities.

Reasoning Compression. Several recent methods aim to shorten reasoning traces while preserving accuracy. GFPO (Shrivastava et al., 2025) filters training rollouts by length and token efficiency within enlarged sample groups, reducing GRPO’s length inflation by 46–85% on STEM and coding benchmarks. OPSDC (Sang et al., 2026) uses a conciseness instruction as privileged context, achieving 57–59% token reduction on MATH-500. ConPress (Deng et al., 2026) exploits multi-question contextual pressure to elicit self-compressed traces, yielding 30–60% reductions via SFT. Accordion-Thinking (Yang et al., 2026) trains models to produce periodic step summaries that can be discarded, achieving 3× throughput with a gap-vanishing phenomenon during RL training. CEEH (Luo et al., 2026) proposes difficulty-aware entropy regularization to prevent entropy collapse under length constraints.

Our analysis complements these efforts by showing that compression which suppresses uncertainty expression, even when producing correct answers, can harm OOD generalization, and that this effect is modulated by task coverage. Therefore, the key challenge is to filter out non-informative content while retaining epistemic expressions that enable iterative belief refinement, rather than blindly compressing the reasoning process.

B Additional Analysis of Epistemic Tokens

B.1 Per-Token Analysis of Epistemic Verbalization

In Table 1 of Section 3, we compared the average number of ten epistemic tokens per response. Figure AA further extends this analysis by examining how the average per-response count of each individual token changes under varying levels of conditioning information. When examining the per-token counts, all tokens exhibit a consistent trend:

$$\mathbb{E}[E(y)]_{(1)} > \mathbb{E}[E(y)]_{(3)} > \mathbb{E}[E(y)]_{(4)} > \mathbb{E}[E(y)]_{(2)}, \quad (5)$$

where tokens such as *wait*, *maybe*, and *perhaps* are particularly prominent.

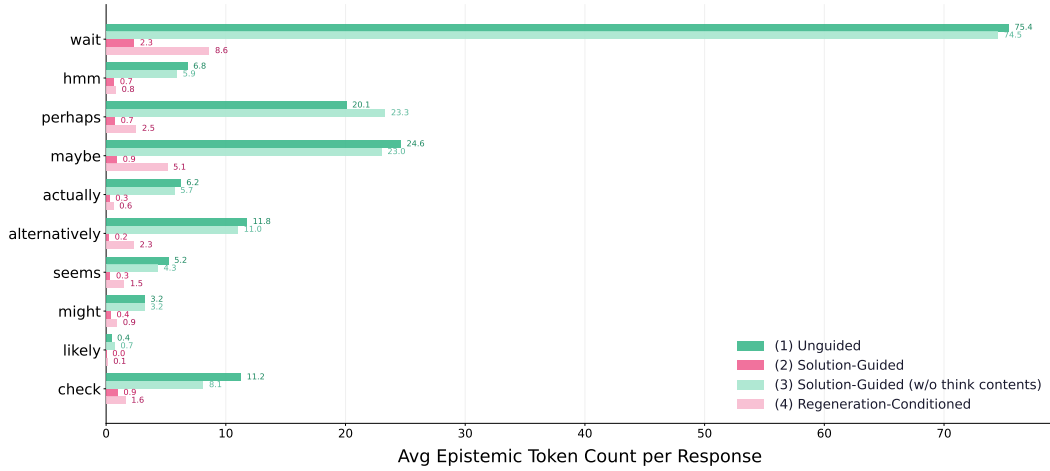


Figure 9: Per-token breakdown of epistemic token usage across the four generation settings. Each bar represents the average number of occurrences per response for an individual epistemic token. All tokens follow the same ordering as the aggregate trend, with *wait*, *maybe*, and *perhaps* showing the largest variation across settings.

B.2 Comparison of Epistemic Token Usage Across Models

Following the analysis of DeepSeek-R1-Distill-Qwen-7B (DeepSeek-Distill-7B) in Section 3, we further compare epistemic token usage across three settings: DeepSeek-Distill-7B, Qwen3-8B with thinking mode enabled, and Qwen3-8B with thinking mode disabled.

As shown in Figure 10, both DeepSeek-Distill-7B and Qwen3-8B with thinking mode enabled produce substantially more epistemic tokens than Qwen3-8B with thinking mode disabled. While the two thinking-enabled models share a similar tendency to express uncertainty, they differ in their preferred epistemic tokens. For instance, DeepSeek-Distill-7B frequently uses *wait* and employs *perhaps* and *maybe* at comparable rates, whereas Qwen3-8B uses *perhaps* relatively less and favors *maybe*. Qwen3-8B also uses *alternatively* and *check* far more than DeepSeek-Distill-7B, and overall embeds a greater amount of uncertainty within its reasoning.

Extending the discussion in Section 3, we also observe that Qwen3-8B generates far fewer epistemic tokens under solution-guided generation than under unguided generation. Across all three settings, Qwen3-8B with thinking mode enabled produces the most epistemic tokens, followed by DeepSeek-Distill-7B, and then Qwen3-8B with thinking mode disabled.

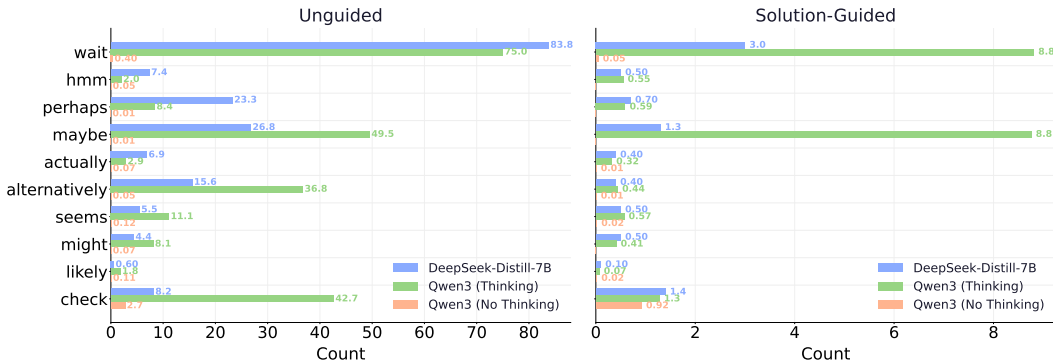


Figure 10: Comparison of epistemic token usage across DeepSeek-R1-Distill-Qwen-7B, Qwen3-8B (thinking enabled), and Qwen3-8B (thinking disabled).

B.3 Analysis of GRPO and SDPO Reasoning Patterns by Task Coverage

To provide a more in-depth analysis of the results in Figure 8 (Section 6.2), we compare the change in epistemic token counts relative to the base model on AIME24 across six training configurations: GRPO and SDPO, each with $|D| \in \{1, 64, 512\}$. Figure 11 shows that GRPO tends to increase epistemic token usage relative to the base model as $|D|$ grows, whereas SDPO reduces epistemic token usage, with smaller $|D|$ leading to a greater reduction.

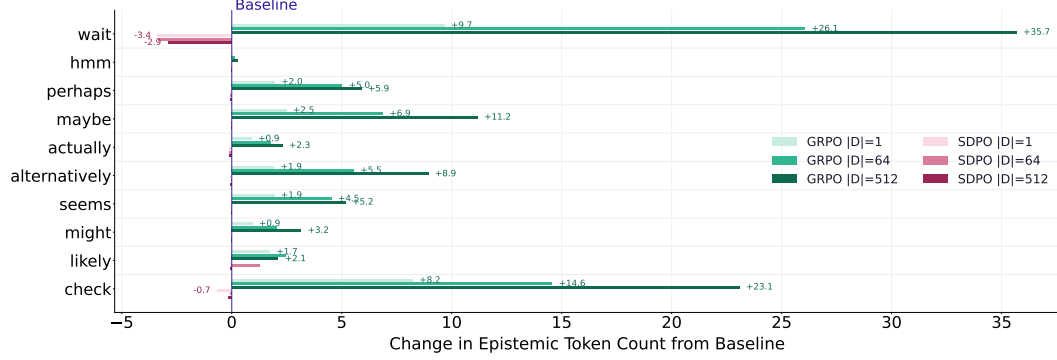


Figure 11: Change in epistemic token counts relative to the base model on AIME24 for GRPO and SDPO with $|D| \in \{1, 64, 512\}$.

B.4 Comparative Analysis of Epistemic and Overall Word Usage

In addition to analyzing epistemic tokens, we examine how the overall word distribution in model responses shifts across the base model, GRPO-trained model, and SDPO-trained model. We first extract English words from each response using regular expressions and construct a reference vocabulary $\mathcal{V}$ from all unique words appearing in the base model responses. For each word $w \in \mathcal{V}$ and model $m \in \{\text{base, GRPO, SDPO}\}$, we compute the average per-response frequency:

$$\mu_m(w) = \frac{1}{N} \sum_{i=1}^N f(w, r_i^{(m)}), \quad (6)$$

where $f(w, r)$ denotes the number of occurrences of word w in response r . We then define the frequency shift $\Delta_m(w) = \mu_m(w) - \mu_{\text{base}}(w)$ to capture how much each word’s usage changes relative to the base model.

Using these frequency shifts, we compare the behavior of the predefined set of epistemic tokens T (as defined in Section 3) against the rest of the vocabulary. Specifically, we compare $\{\Delta_m(w) \mid w \in T\}$ against $\{\Delta_m(w) \mid w \in \mathcal{V} \setminus T\}$ to determine whether the changes in epistemic token usage are disproportionately large relative to the overall shift in word distribution. This allows us to distinguish whether the training procedure specifically amplifies epistemic expressions or merely induces a uniform change across the vocabulary.

As shown in Figure 12, the disparity between the two panels confirms that the observed epistemic shifts are not byproducts of general vocabulary drift. Across all model-method pairs, the average per-word shift over the full vocabulary remains near zero ($|\Delta| < 1$), whereas epistemic tokens exhibit shifts up to $30\times$ – $40\times$ larger in magnitude (e.g., -11.9 and -12.2 for SDPO). This indicates that both training objectives selectively target epistemic expressions rather than uniformly perturbing the output distribution.

B.5 Uncertainty Annotation via LLM-as-Judge

We employed epistemic markers as indicators of uncertainty expressions, as they serve as practical and interpretable cues for regions where uncertainty externalization is likely to

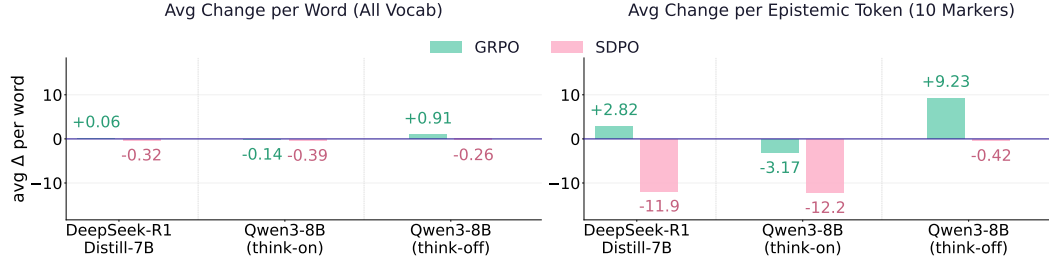


Figure 12: Comparison of per-word frequency shifts ($\Delta_m(w)$) for epistemic tokens (T) versus non-epistemic tokens ($V \setminus T$) in GRPO- and SDPO-trained models relative to the base model. Epistemic tokens exhibit disproportionately large frequency changes compared to the general vocabulary, indicating that the training procedures specifically amplify or suppress epistemic expressions rather than inducing uniform changes across the word distribution.

occur. To more comprehensively capture diverse forms of uncertainty expression, we additionally leveraged GPT-5.4 (Singh et al., 2025) as an automated annotator. Specifically, GPT-5.4 was prompted to identify and count uncertainty expressions in each model-generated response. The prompt used for this annotation is shown below.

Prompt for Uncertainty Annotation
You are an expert linguistic annotator. Your task is to count uncertainty expressions in a model-generated math reasoning response. Uncertainty expressions include: • Hedging expressions: expressions that indicate uncertainty (e.g., <i>wait, hmm, perhaps, maybe</i>) • Doubt markers: expressions that explicitly convey uncertainty (e.g., <i>I'm not sure, not certain, double-check, verify this</i> when used to express doubt) Do NOT count: • Confident statements such as "we know", "clearly" • Mathematical conditionals such as "if $x = 0$, then..." Return ONLY a JSON object with exactly these fields: <pre>{ "count": <integer, total number of uncertainty expressions>, "examples": [<up to 5 short quoted phrases from the text>] }</pre>

Table 4: Representative examples of uncertainty expressions identified by GPT-5.4. Expressions are drawn from model-generated reasoning traces across all evaluated methods.

Uncertainty Expression
"maybe there's a simpler way"
"I think the key here is to find s and t first, right?"
"perhaps there's a relationship here"
"perhaps I made a miscalculation"
"Wait, but this might get complicated"
"maybe we can assume some symmetry"
"maybe we don't consider $n = 0$ "
"might not satisfy the surface area constraint"
"actually, no. Let me think again."
"Wait, hold on. Is that correct?"
"I'm not entirely sure"
"Hmm, that seems contradictory"
"it seems a bit tricky"
"it seems less likely"
"Alternatively, perhaps it's better to think of it..."
"maybe I need another approach"

Table 4 presents representative examples of uncertainty expressions identified by the annotator. The majority of identified expressions coincide with the epistemic tokens examined in the previous analysis (e.g., *maybe, perhaps, wait*), while others manifest as multi-token phrases such as "I'm not entirely sure" that cannot be captured by single-token frequency alone. This confirms that epistemic markers serve as a reliable proxy for uncertainty exter-

nalization, while the LLM-as-Judge approach provides complementary coverage over more diverse expression forms.

Table 5: Average number of uncertainty expressions per response, as annotated by GPT-5.4.

Method	Avg. Unc. Expr.
Baseline	21.36
GRPO	24.36
SDPO	9.18

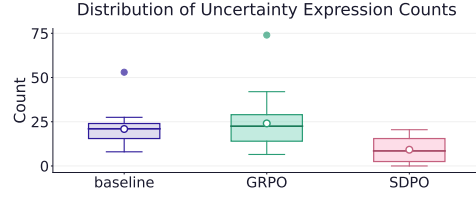


Figure 13: Box plot of uncertainty expression counts across methods.

Table 5 reports the average number of such expressions per response for each method. When evaluated through LLM-as-Judge annotation, SDPO again substantially reduces uncertainty expressions compared to both the baseline and GRPO, consistent with the token-level analysis. Figure 13 complements this with a box plot comparison across methods: notably, GRPO exhibits considerably greater variance than the baseline, suggesting that RLVR can amplify inconsistency in hedging behavior. In contrast, SDPO maintains consistently low counts with a narrow spread, confirming that on-policy self-distillation suppresses unnecessary hedging and self-doubt during reasoning.

B.6 Student–Teacher KL Divergence on Epistemic Tokens

We additionally measure the per-token KL divergence between the student $\pi_\theta(\cdot \mid x, y_{<t})$ and the self-teacher $\pi_\theta(\cdot \mid x, f, y_{<t})$ for each surface-form variant of the epistemic tokens in $\mathcal{T}$. As shown in Table 6, every variant exceeds the corpus-wide mean KL of 0.076, ranging from $1.9\times$ (‘Wait’) to $6.8\times$ (‘Hmm’). Notably, sentence-initial variants (e.g., ‘Hmm’ at $6.8\times$, ‘wait’ at $3.8\times$) tend to exhibit larger divergence than mid-sentence forms, suggesting that the self-teacher most strongly disagrees at positions where the student initiates epistemic deliberation. This provides a quantitative explanation for the suppression of epistemic verbalization under SDPO.

Table 6: KL divergence between student and self-teacher for epistemic tokens $\mathcal{T}$. The corpus-wide mean KL across all 1,642 frequent tokens is 0.076.

Token	Variant	Mean KL	Count	Ratio
hmm	‘Hmm’	0.516	542	$6.8\times$
	‘Hm’	0.337	887	$4.4\times$
maybe	‘Maybe’	0.432	52	$5.7\times$
	‘Maybe’	0.192	937	$2.5\times$
	‘maybe’	0.173	4,676	$2.3\times$
actually	‘actually’	0.188	1,134	$2.5\times$
alternatively	‘Alternatively’	0.194	179	$2.6\times$
	‘Alternatively’	0.175	3,471	$2.3\times$
might	‘might’	0.171	684	$2.3\times$
perhaps	‘Perhaps’	0.255	54	$3.4\times$
	‘perhaps’	0.165	4,173	$2.2\times$
wait	‘wait’	0.285	1,066	$3.8\times$
	‘Wait’	0.196	1,314	$2.6\times$
	‘Wait’	0.146	15,000	$1.9\times$
seems	‘seems’	0.152	904	$2.0\times$

C Can Uncertainty-Encouraging Prompts Mitigate the Suppression?

We further investigate whether prompting the teacher to produce more verbose reasoning with explicit expressions of uncertainty can mitigate the performance degradation caused by the suppression of uncertainty expressions. To this end, we design a modified regeneration prompt that encourages detailed, step-by-step reasoning with uncertainty, and repeat the reasoning behavior analysis (Section 3) and on-policy self-distillation experiments (Section 5) using this prompt.

Prompt for unguided generation	{question} Please reason step by step, and put your final answer within \boxed{ }.
Regeneration prompt (followed the prompt in Hübottter et al. (2026))	{question} Please reason step by step, and put your final answer within \boxed{ }. Correct solution: {previously correct solution} Correctly solve the original question.
Regeneration prompt with encouraging uncertainty	{question} Please reason step by step, and put your final answer within \boxed{ }. Correct solution: {previously correct solution} Solve the original question from scratch with a full, detailed reasoning trace, without skipping any steps, and include as much uncertainty or ambiguity in the explanation.

Table 7: Response characteristics of solution-guided generation with and without the uncertainty-encouraging prompt.

	Avg. Length	Epistemic Token Count
Unguided	13,054	182.5
Solution-Guided (vanilla)	1,873	8.8
Solution-Guided (w/ uncertainty encouraging)	3,098	31.4

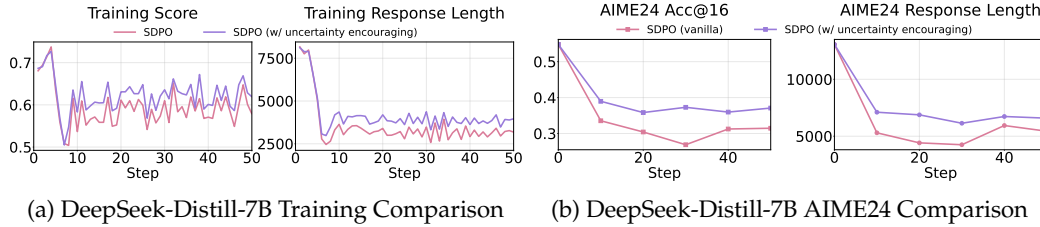


Figure 14: Training dynamics and AIME24 evaluation of on-policy self-distillation using the uncertainty-encouraging regeneration prompt. (a) Training score and response length over training steps. (b) AIME24 Acc@16 and response length over training steps. While the uncertainty-encouraging prompt partially mitigates response shortening, the overall performance degradation persists.

Table 7 shows that the uncertainty-encouraging prompt partially recovers the suppressed reasoning behavior: compared to the vanilla solution-guided prompt, it produces longer responses (3,098 vs. 1,873 tokens) and more epistemic expressions (31.4 vs. 8.8). However, both metrics remain far below those of unguided generation (13,054 tokens, 182.5 epistemic tokens), indicating that the presence of a reference solution fundamentally constrains the model’s reasoning verbosity regardless of the prompt instruction.

Figure 14 shows that the uncertainty-encouraging prompt leads to slightly higher training scores and longer responses compared to the vanilla setting, and this marginal improvement carries over to AIME24 evaluation. Nevertheless, both variants still suffer from substantial performance degradation, suggesting that prompting alone is insufficient to resolve the fundamental issue of uncertainty suppression in solution-guided generation.

D Experimental Details

Training For GRPO and SDPO training, we built upon the SDPO implementation (<https://github.com/lasgroup/SDPO>) and additionally incorporated the DAPO-Math-17k dataset. The original DAPO-Math-17k dataset uses the following prompt format:

Solve the following math problem step by step. The last line of your response should be of the form Answer: \$Answer (without quotes) where \$Answer is the answer to the problem.\n\n{question}\nRemember to put your answer on its own line after "Answer:".

We replaced this with a simpler format:

{question}\nPlease reason step by step, and put your final answer within \boxed{ }.

as we observed that this format consistently yielded higher evaluation performance. For reward verification, we used the scoring function from the verl framework, which extracts the answer from the `\boxed{ }` expression and verifies correctness via exact match followed by mathematical equivalence checking using math-verify, adapted from EleutherAI’s Im-evaluation-harness (Gao et al., 2024).

The GRPO and SDPO training hyperparameters are listed in Tables 8, 9, and 10. For the experiments on the relationship between task coverage and learning performance in Figure 8, we reduced the question batch size to 64 due to the use of smaller training questions.

Table 8: Common hyperparameters shared by GRPO and SDPO.

Category	Parameter	Value
Data	Max. prompt length	2048
	Max. response length	20480
Batching	Question batch size	256
	Mini batch size	64 or 128
	Number of rollouts	8
Rollout	Inference engine	vllm
	Temperature	1.0
Training	Optimizer	AdamW
	Warmup steps	10
	Weight decay	0.01
	Gradient clip norm	1.0

Table 9: GRPO-specific hyperparameters.

Category	Parameter	Value
Loss	ϵ -high	0.28
	Rollout IS clip	2
	KL coefficient (λ)	0.0
Training	Learning rate	1×10^{-6}

Evaluation We evaluated all models using the hyperparameters listed in Table 11, following the recommended settings from each model’s official documentation.^{1,2}

¹<https://huggingface.co/Qwen/Qwen3-8B>

²<https://huggingface.co/allenai/OLMo-3-7B-Instruct>

Table 10: SDPO-specific hyperparameters.

Category	Parameter	Value
Loss	Distillation divergence	Jensen-Shannon
	Top-K distillation	100
	EMA update rate	0.0
Training	Learning rate	1×10^{-5}

Table 11: Evaluation hyperparameters.

Model	Max Tokens	Temp.	Top- p	Top-K
DeepSeek-R1-Distill-7B	38912	0.6	0.95	20
Qwen3-8B (thinking)	38912	0.6	0.95	20
Qwen3-8B (non-thinking)	38912	0.7	0.8	20
OLMo-3-7B-Instruct	38912	0.6	0.95	20

Chat Templates for Different Model Series Table 12 summarizes the chat template formats used by several open-weight model families, each using distinct special tokens to delineate user and assistant turns. The placeholder {prompt} denotes the user message.

Table 12: Chat template formats by model series.

Model	Template
DeepSeek-R1-Distill-7B	< begin_of_sentence >< User > {prompt} < Assistant ><think>
Qwen3-8B (think ON)	< im_start >user\n {prompt} < im_end > < im_start >assistant
Qwen3-8B (think OFF)	< im_start >user\n {prompt} < im_end > < im_start >assistant\n<think>\n</think>
OLMo-3-7B-Instruct	< im_start >system\n {sys} < im_end > < im_start >user\n {prompt} < im_end >\n< im_start >assistant

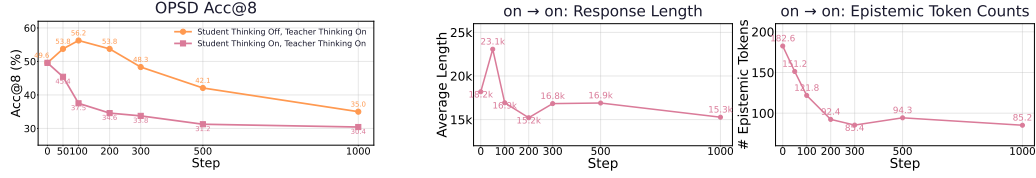
E Comparison with OPSD

Recently, OPSD (Zhao et al., 2026) demonstrated performance gains in mathematical reasoning through self-distillation, particularly on the Qwen3 series. Unlike our setup, where both the student and the teacher either enable or disable thinking mode, OPSD adopts a hybrid configuration in which the student operates with thinking mode disabled while the teacher has it enabled.

As our experiments also confirm, enabling thinking mode produces substantially longer responses with a greater number of epistemic tokens, making this hybrid setup function more akin to conventional teacher-student distillation, despite using the same underlying model. We note that this configuration is inherently limited to model families such as Qwen3 that support toggling thinking mode on and off.

Furthermore, for training efficiency, OPSD does not train on the entire student response; instead, it focuses only on a prefix (1024 tokens by default). Unlike SDPO, which performs full fine-tuning based on verl (Sheng et al., 2024), OPSD uses LoRA fine-tuning based on trl (von Werra et al., 2020). Additionally, OPSD’s hyperparameters (batch size 32, learning rate $1e-6$) are smaller than those used in our setup (batch size 256, learning rate $1e-5$), resulting in higher training efficiency but smaller parameter updates per step.

Under this hybrid setup with prefix learning in Qwen3-1.7B, as shown in Figure 15a (orange line), we observe an intriguing training dynamic: in the early stages, the thinking-enabled



(a) Hybrid vs. homogeneous setup performance over training.

(b) Response length and epistemic token usage in the homogeneous setup.

Figure 15: Training dynamics of OPSD hybrid distillation in Qwen3-1.7B compared to our homogeneous chat template setup. (a) Under the hybrid setup, the thinking-enabled teacher initially improves student performance, but gains reverse over time. In contrast, the homogeneous setup shows a consistent decrease. (b) Response length and epistemic token usage in the homogeneous setup.

teacher drives the student toward longer responses with improved performance, demonstrating the effectiveness of hybrid distillation in the early phase of training. However, as training progresses, the response length gradually decreases, accompanied by a corresponding degradation in performance. In contrast, under our homogeneous setup as in Figure 15b, where both the student and the teacher have thinking mode enabled, performance declines consistently, while response length and epistemic token count also decrease steadily, consistent with our earlier analysis.

While this hybrid distillation setup for the Qwen3 series presents an interesting research direction with its own unique training dynamics, such as why performance initially improves before declining and whether this stems from changes in reasoning behavior or chat template mismatch, a thorough investigation is beyond the scope of this work and is left for future exploration.

F More On-Policy Self-Distillation Results

F.1 Qwen3-8B (Thinking Mode: OFF)

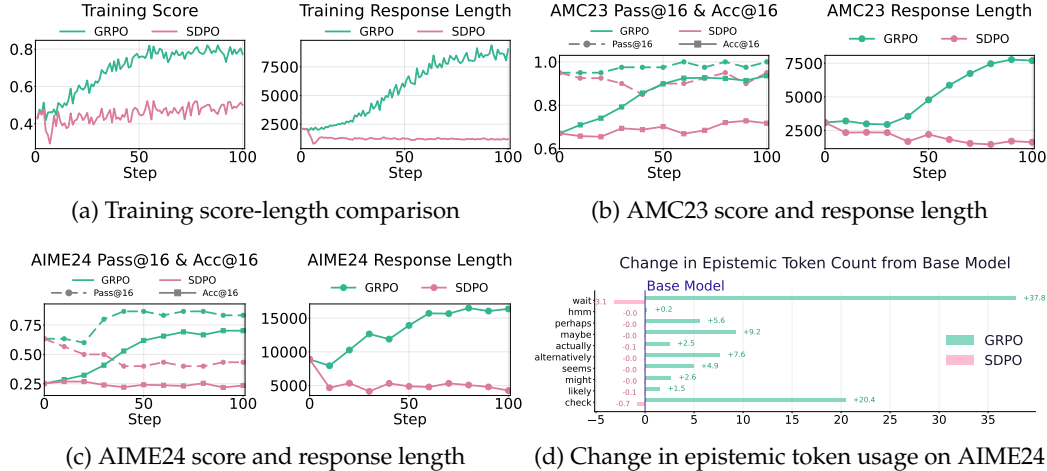


Figure 16: Extended results for Qwen3-8B (thinking mode off): training score-length trade-off, evaluation performance on AMC23 and AIME24, and change in epistemic token usage.

As an extension of Figure 5 in Section 5.3, we additionally compare AMC23 evaluation scores and the change in epistemic token usage on AIME24. As shown in Figure 16, GRPO significantly increases response length and yields substantial performance gains on both benchmarks. In contrast, SDPO exhibits divergent trends: on AMC23, acc@16 increases from

0.67 to 0.73 while reducing response length by roughly half, whereas on AIME24, acc@16 slightly decreases from 0.25 to 0.23, with pass@16 dropping more substantially. Notably on AMC23, SDPO achieves approximately a 6-point improvement with shorter responses, whereas GRPO obtains a much larger improvement of around 36 points at the cost of considerably longer responses. Achieving large performance gains while maintaining a reasonable response length remains an open challenge.

F.2 Qwen3-1.7B (Thinking Mode: ON)

To investigate whether our findings generalize to smaller models, we conducted the same on-policy self-distillation experiment on Qwen3-1.7B. As shown in Figure 17, our finding that self-distillation reduces uncertainty and makes outputs more concise, while simultaneously degrading mathematical reasoning performance, remains consistent across model sizes. Notably, the performance degradation is more severe for the 1.7B model (45.9%) compared to Qwen3-8B with Thinking Mode ON (12.1%) and DeepSeek-R1-Distill-Qwen-7B (40.7%). This suggests that the externalization of uncertainty may be particularly important for robust reasoning in smaller models, which possess more limited inherent capabilities.

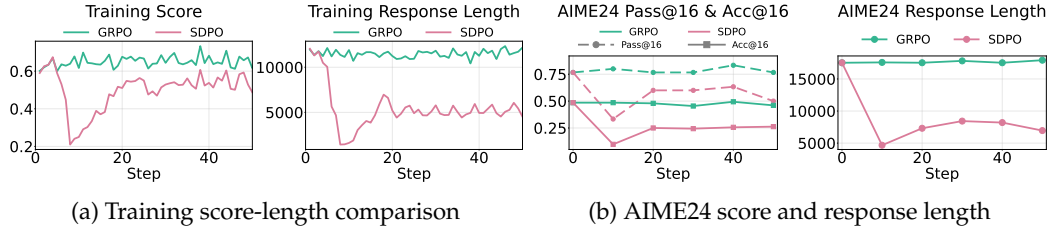


Figure 17: GRPO and SDPO results on Qwen3-1.7B (Thinking Mode: ON): training score and response length, and OOD evaluation on AIME24. Self-distillation consistently reduces response length but leads to a substantial drop in performance, with a more pronounced degradation compared to larger models.

F.3 Olmo-3-7B-Instruct

In addition to DeepSeek-R1-Distill-Qwen-7B and Qwen3-8B, we further evaluate on-policy self-distillation on OLMo-3-7B-Instruct from a different model family. As shown in Figure 18, consistent with our previous analyses, SDPO also degrades reasoning performance on this model, with OOD evaluation scores falling below those of the base model. This confirms that our findings are not model-dependent but reflect robust characteristics of reasoning behavior across diverse model families.

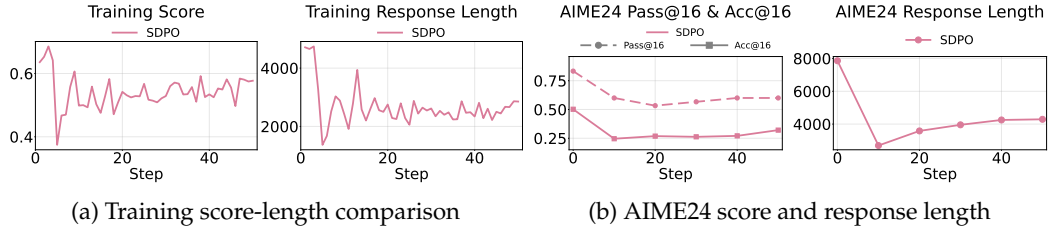


Figure 18: SDPO results on OLMo-3-7B-Instruct: training score and response length, and OOD evaluation on AIME24.

F.4 Pass@16 Score

In addition to the acc@16 scores for DeepSeek-Distill-7B and Qwen3-8B (thinking mode enabled) presented in Figures 3b, 3c, 4b, and 4c, we report pass@16 scores in Figure 19. The gap between GRPO and SDPO in pass@16 is larger for DeepSeek-Distill-7B than for Qwen3-8B, and more pronounced on the harder benchmark (AIME24) compared to AMC23.

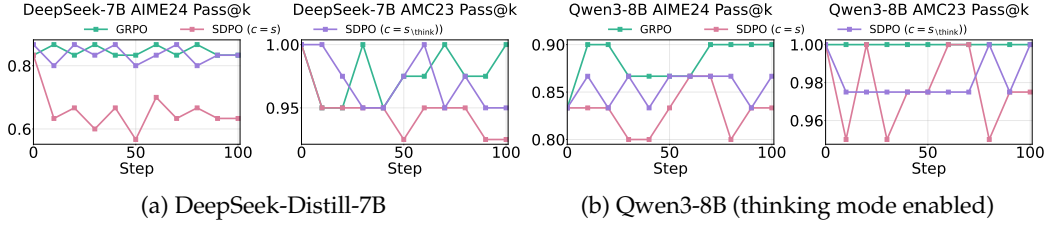
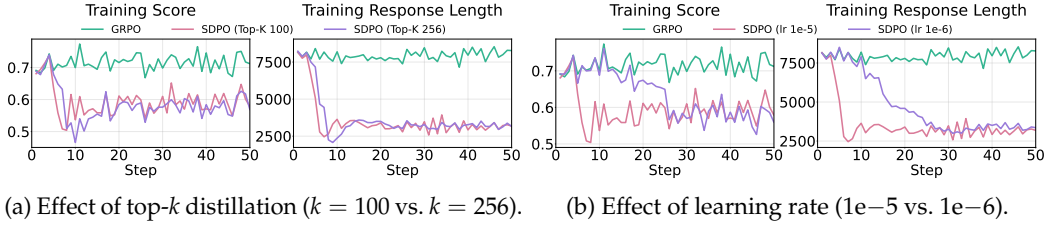


Figure 19: Pass@16 on AMC23 and AIME24 for GRPO and SDPO across training steps.

G More Ablation Study

To examine the effect of various training hyperparameters on self-distillation behavior, we conduct additional experiments by varying the top- k distillation parameter and the learning rate. As shown in Figure 20a, increasing top- k from 100 to 256 yields no significant difference in training dynamics or final performance. Meanwhile, as illustrated in Figure 20b, reducing the learning rate from $1e-5$ to $1e-6$ merely slows the rate of degradation; the model ultimately converges to the same reasoning behavior.

Figure 20: Ablation study on top- k distillation and learning rate. Both modifications fail to prevent the convergence toward degraded reasoning behavior; a lower learning rate only delays the process.

H Qualitative Analysis

H.1 Unguided vs. Solution-Guided Generation

We analyze examples of $\mathcal{D}_{\text{ug}}$ (Unguided responses) and $\mathcal{D}_{\text{sg}}$ (Solution-guided responses) introduced in Section 4. Below, we present the two responses generated for the same problem. As illustrated, the unguided response exhibits substantially more uncertainty and exploratory reasoning, frequently revisiting assumptions, considering alternative approaches, and self-correcting along the way. In contrast, the solution-guided response proceeds in a more concise and direct manner, arriving at each intermediate result without detours.

Problem. How many ways can 8 mutually non-attacking rooks be placed on a 9×9 chessboard so that all 8 rooks are on squares of the same color? (Two rooks are said to be attacking each other if they are placed in the same row or column of the board.) Please reason step by step, and put your final answer within $\square$.

Unguided Response – Exploratory Reasoning

Alright, so I have this problem where I need to figure out how many ways I can place 8 mutually non-attacking rooks on a 9×9 chessboard, and all of them have to be on squares of the same color. Hmm, let me break this down.
First, I know that a chessboard is 8×8 , but this is a 9×9 chessboard. So, it's one square larger on each side. Also, non-attacking rooks mean that no two rooks can be in the same row or

column. Since it's 9×9 , if I were placing 9 rooks, they would each occupy their own row and column. But here, we're placing 8 rooks, so one row and one column will be empty. But wait, the problem also says all rooks must be on squares of the same color. So, not only do they have to not attack each other, but they all have to be on, say, white squares or all on black squares.

[...]

So, perhaps, the number of ways to place 8 non-attacking rooks on white squares would be equal to the number of permutation matrices on the white squares.

Similarly for black squares.

But how do we compute that?

I recall that in combinatorics, the number of permutation matrices avoiding certain positions is given by the inclusion-exclusion principle, but that might get complicated.

Alternatively, maybe we can model this as a bipartite graph matching problem.

[...]

But I don't know the exact formula for that.

Alternatively, perhaps we can use the principle of inclusion-exclusion or recursion.

Alternatively, perhaps notice that the white squares form a bipartite graph which is a union of two complete bipartite graphs, but that might not hold.

Wait, let me consider the structure of the white squares. [...] So, the bipartite graph is actually composed of two complete bipartite graphs: one connecting even rows to even columns, and the other connecting odd rows to odd columns.

Therefore, the bipartite graph is actually two disjoint complete bipartite graphs: one between the odd rows (5 rows) and odd columns (5 columns), and another between the even rows (4 rows) and even columns (4 columns).

So, the white squares form two separate complete bipartite graphs: $K_{5,5}$ and $K_{4,4}$.

[...]

So, to get a matching of size 8, we need to have k edges from $K_{5,5}$ and $8 - k$ edges from $K_{4,4}$, where k ranges from 4 to 5.

If $k = 4$, then $8 - 4 = 4$ edges from $K_{4,4}$.

If $k = 5$, then $8 - 5 = 3$ edges from $K_{4,4}$.

[...]

So, for $K_{5,5}$, the number of matchings of size 4 is $\binom{5}{4}^2 \cdot 4! = 25 \times 24 = 600$.

Similarly, the number of matchings of size 5 in $K_{5,5}$ is $\binom{5}{5}^2 \cdot 5! = 1 \times 120 = 120$.

For $K_{4,4}$, the number of matchings of size 4 is $\binom{4}{4}^2 \cdot 4! = 1 \times 24 = 24$.

And the number of matchings of size 3 is $\binom{4}{3}^2 \cdot 3! = 16 \times 6 = 96$.

Therefore: $600 \times 24 + 120 \times 96 = 14,400 + 11,520 = 25,920$.

[...]

In $K_{5,4}$, the number of perfect matchings (size 4) is $\binom{5}{4} \cdot \binom{4}{4} \cdot 4! = 5 \times 1 \times 24 = 120$.

Similarly, in $K_{4,5}$, the number of perfect matchings is $\binom{4}{4} \cdot \binom{5}{4} \cdot 4! = 1 \times 5 \times 24 = 120$.

Therefore, the number of matchings of size 8 in the black square bipartite graph is $120 \times 120 = 14,400$.

Therefore, total is $25,920 + 14,400 = 40,320$.

But wait, $40,320$ is $8! = 40,320$. Hmm, that's interesting.

But why?

Is there a combinatorial interpretation where the number of ways is $8!$ multiplied by something?

Wait, $8!$ is 40320 , which is exactly the number of ways we got.

But why is it exactly $8!$? It seems coincidental.

[...]

Wait, perhaps it's best to just accept that the total number is $40,320$.

So, the final answer is $40,320$.

Solution-Guided Response – Concise, Direct Reasoning

Alright, so I have this problem where I need to figure out how many ways I can place 8 mutually non-attacking rooks on a 9×9 chessboard, and all of them have to be on squares of the same color. Hmm, okay, let's break this down step by step.

First, I know that a rook can move any number of squares vertically or horizontally, so non-attacking rooks must be placed on different rows and different columns. That means for 8 rooks, we need 8 distinct rows and 8 distinct columns.
 [...] Specifically, there are 41 squares of one color and 40 of the other. [...] So white squares are 41 and black squares are 40.
 So, I need to calculate the number of ways to place 8 non-attacking rooks on white squares and then on black squares separately, and then add them together.
 Starting with white squares. [...] The white square bipartite graph has two components: $K_{5,5}$ and $K_{4,4}$. Similarly, [...] the black square bipartite graph has two components: $K_{5,4}$ and $K_{4,5}$.
 [...] The number of ways to choose 4 edges from $K_{5,5}$ is $\binom{5}{4}^2 \cdot 4! = 25 \times 24 = 600$. The number of ways to choose 5 edges from $K_{5,5}$ is $\binom{5}{5}^2 \cdot 5! = 1 \times 120 = 120$. Similarly, for $K_{4,4}$, the number of ways to choose 4 edges is $\binom{4}{4}^2 \cdot 4! = 1 \times 24 = 24$, and choosing 3 edges is $\binom{4}{3}^2 \cdot 3! = 16 \times 6 = 96$.
 So, the total number of ways for white squares is $(600 \times 24) + (120 \times 96) = 14,400 + 11,520 = 25,920$.
 For black squares, [...] the total number of ways is 14,400.
 Adding both cases together, we get $25,920 + 14,400 = \boxed{40,320}$.

H.2 GRPO vs. SDPO Generation

Problem (AIME24). Find the number of ways to place a digit in each cell of a 2×3 grid so that the sum of the two numbers formed by reading left to right is 999, and the sum of the three numbers formed by reading top to bottom is 99. The grid below is an example of such an arrangement because $8 + 991 = 999$ and $9 + 9 + 81 = 99$:

0	0	8
9	9	1

The correct answer is $\boxed{45}$.

GRPO explores with repeated reflections, deriving the extra constraint $a + b + c = 8$ beyond the pairwise sums, and correctly counts $\binom{10}{2} = 45$ solutions. SDPO proceeds directly and stops after deriving $a + d = b + e = c + f = 9$, multiplying $10 \times 10 \times 10 = 1000$ without checking the row-sum equation, thereby missing the additional constraint $a + b + c = 8$.

DeepSeek-R1-Distill-Qwen-7B GRPO Generation (Correct: 45)

Okay, so I need to figure out how many ways there are to fill a 2×3 grid with digits such that two conditions are satisfied: (1) the sum of the two numbers formed by reading left to right is 999, and (2) the sum of the three numbers formed by reading top to bottom is 99.

Let me visualize the grid as

a	b	c
d	e	f

. The first condition gives $(100a + 10b + c) + (100d + 10e + f) = 999$, and the second gives $(10a + d) + (10b + e) + (10c + f) = 99$.

[...]

So we have:

Equation (A): $100S_1 + 10S_2 + S_3 = 999$, where $S_1 = a + d$, $S_2 = b + e$, $S_3 = c + f$.

Equation (B): $10T + U = 99$, where $T = a + b + c$, $U = d + e + f$.

[...] Since S_1 is $a + d$, S_1 can be from 0 to 18. But if S_1 is 10, then $100S_1 = 1000$, which is already larger than 999. Therefore, S_1 can be from 0 to 9.

For $S_1 = 9$: $100 \cdot 9 = 900$. Then $10S_2 + S_3 = 99$.

For $S_1 = 8$: $100 \cdot 8 = 800$. Then $10S_2 + S_3 = 199$. But $10S_2 + S_3$ can be at most $10 \cdot 18 + 18 = 198$. So 199 is too big. Therefore S_1 can't be 8.

For $S_1 = 9$, $10S_2 + S_3 = 99$. Since S_2 and S_3 are between 0 and 18, $10S_2 + S_3$ can be between 0 and 198. So 99 is possible.

[...] Therefore S_2 must be 9. Hence, $S_2 = 9$, then $S_3 = 99 - 90 = 9$.

Let me check other S_1 values. For $S_1 = 7$: $10S_2 + S_3 = 299$, even larger. Hence impossible. Similarly for lower S_1 . Hence, only $S_1 = 9$ is possible.

Wait, let me double-check that $S_1 = 9$ is the only possible value. For $S_1 = 9$, we have $10S_2 + S_3 = 99$, which requires $S_2 = 9$ and $S_3 = 9$. [...] So only $S_2 = 9$ and $S_3 = 9$. Therefore, Equation (A) only holds when $S_1 = S_2 = S_3 = 9$.
 [...] From $T + U = 27$ and $10T + U = 99$: subtracting gives $9T = 72$, so $T = 8, U = 19$.
 [...] Substituting $d = 9 - a, e = 9 - b, f = 9 - c$, Equation 5 ($d + e + f = 19$) is automatically satisfied. The main constraint reduces to $a + b + c = 8$ with $a, b, c \in \{0, \dots, 9\}$.
 Since $8 < 9$, the upper bound is automatic, so the count is the number of non-negative integer solutions to $a + b + c = 8$, which is $\binom{10}{2} = 45$.
 Since $d = 9 - a$ and $a \leq 8, d$ ranges from 1 to 9 — all valid digits. Similarly for e, f . Therefore, all 45 solutions are valid.
Wait, let me check if there are constraints I missed. When forming the numbers N_1 and N_2 , they can have leading zeros, which is fine since the example itself has 008 in the top row. [...] So there is no problem with leading zeros.
 Let me verify with the given example: $a = 0, b = 0, c = 8$, so $d = 9, e = 9, f = 1$. Columns: $09 + 09 + 81 = 99 \checkmark$. Rows: $008 + 991 = 999 \checkmark$.
 Another example: $a = 8, b = 0, c = 0$. Then $d = 1, e = 9, f = 9$. Top: 800, bottom: 199. $800 + 199 = 999 \checkmark$. Columns: $81 + 9 + 9 = 99 \checkmark$.
 Another: $a = 0, b = 1, c = 7$. Then $d = 9, e = 8, f = 2$. $17 + 982 = 999 \checkmark$. $9 + 18 + 72 = 99 \checkmark$.
Wait, one more check — what if leading zeros aren't allowed? Then the top row would need $a \neq 0$, but the example given has $a = 0$, so that interpretation is invalid. Therefore, leading zeros are allowed, and the answer is indeed 45.
 Therefore, I think the correct answer is 45.

DeepSeek-R1-Distill-Qwen-7B SDPO Generation (Incorrect: 1000)

Okay, so I need to figure out how many ways there are to fill a 2×3 grid with digits so that the sum of the two numbers formed by reading left to right is 999, and the sum of the three numbers formed by reading top to bottom is 99.

Labeling the grid as Row 1: a, b, c and Row 2: d, e, f , the two conditions give:

$$\begin{aligned} (1) \quad & 10(a + b + c) + (d + e + f) = 99 \\ (2) \quad & 100(a + d) + 10(b + e) + (c + f) = 999 \end{aligned}$$

Let $S = a + b + c$, $T = d + e + f$, and $x = a + d$, $y = b + e$, $z = c + f$. Then (1) becomes $10S + T = 99$ and (2) becomes $100x + 10y + z = 999$.

Since $x + y + z = S + T$, substituting $T = 99 - 10S$ gives $x + y + z = 99 - 9S$. Subtracting this from (2):

$$99x + 9y = 900 + 9S \Rightarrow 11x + y = 100 + S.$$

[...] From $0 \leq 99 - 10S \leq 27$ we get $S \in \{8, 9\}$.

Case 1: $S = 8$. Then $T = 19$, $11x + y = 108$, and $x + y + z = 27$. Substituting $y = 108 - 11x$ gives $z = 10x - 81$. The constraint $0 \leq z \leq 18$ forces $x = 9$, so $y = 9, z = 9$. Therefore $a + d = 9, b + e = 9, c + f = 9$.

For each of the three pairs, there are 10 possibilities (since for each digit from 0 to 9, there's exactly one digit that adds up to 9). So for each of the three pairs, there are 10 possibilities. Therefore, the total number of solutions in this case is $10 \times 10 \times 10 = 1000$.

Case 2: $S = 9$. Then $T = 9$, $11x + y = 109$, and $x + y + z = 18$. Substituting gives $z = 10x - 91$. The constraint $0 \leq z \leq 18$ forces $x = 10$, but then $y = 109 - 110 = -1$, which is invalid. Therefore, this case is invalid.

Thus, only Case 1 is valid, leading to 1000 possible configurations.

The digits a, b, c, d, e, f must be digits between 0 and 9. In Case 1, we have $a + d = 9$, so each pair (a, d) can be any pair of digits adding to 9, which is valid. Similarly for (b, e) and (c, f) . So there are no invalid digit values here. Therefore, all 1000 configurations are valid.

Let me check with the example given in the problem:

0	0	8
9	9	1

. Horizontal sum: $008 + 991 = 999 \checkmark$. Vertical sum: $09 + 09 + 81 = 99 \checkmark$. Each of the pairs $(a, d) = (0, 9)$, $(b, e) = (0, 9)$, $(c, f) = (8, 1)$ adds to 9, which fits Case 1. So this example is counted in the 1000 configurations.

Hence, the answer should be 1000.